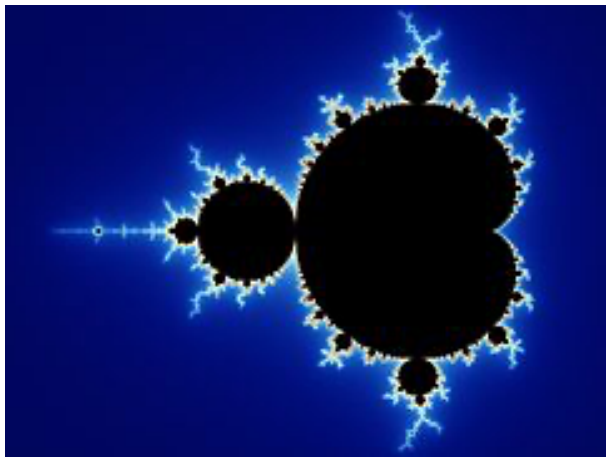
Multicore Programming: Project

JMCS 2024

Parallel fractal generator

The Mandelbrot set[1] is the set of values of *c* in the complex plane for which the orbit of 0 under iteration of the quadratic map $z_{n+1} = z_n^2 + c$ remains bounded. It can be used to produce complex fractal shapes as shown below:



Since every point is computed independently from the others, it is easy to parallelize the computation of the fractal shapes.

The goal of the project is to write a Java application that can compute the Mandelbrot set in parallel and display it. You can use information available online to become familiar with the Mandelbrot set. Please make sure that you indicate your sources if you reuse existing code found on the Web.

Among the many online resources, the site http://java.rubikscube.info/[2] contains very clear explanations and sample code that you can use to get started. A possible approach to develop your solution is to begin with the source code of the 3rd or 4th version of the Applet on that page (`Mandel3.java` or `Mandel4.java`).

You should provide the following files at the end of the project:

- An archive (`.zip` or `.tgz`) that contains the source and class files for a program that will compute and display the Mandelbrot set.
- A short report (2-3 pages) in PDF explaining how you developed and parallelized your program, how to use it, and the speedup you obtained on your computer.

The program should be able to run with a configurable number of threads. It must be executable as a normal program, not just as an Applet (i.e., it must have a `main()` method). It should be able to compute and display the Mandelbrot set at least twice faster with 4 cores than with 1 (ideally it should be around 3 times faster). The faster it goes, the better.

---

[1] https://en.wikipedia.org/wiki/Mandelbrot_set

[2] https://web.archive.org/web/20230929031131/http://java.rubikscube.info/

You are free to use any of the Java concurrency mechanisms for implementing your parallel Mandelbrot program, and you can choose how to split the work among the threads. The program will be judged on the following criteria, in order of importance:

1. Correctness, i.e., threading and synchronization should be done properly.
2. Speed, i.e., how much parallelism can be exploited.
3. Good coding practices, i.e., code is well written and uses the right abstractions.
4. Features, i.e., the program provides basic functionalities with a clean interface.

The report will also be taken into account in the evaluation.

As an example, you can download from the following address a class file with one possible solution of the project (based on the code from http://java.rubikscube.info/):

http://members.unine.ch/pascal.felber/teaching/ConcSys/2024/mandel.zip

The program supports the following commands:

[Left click drag]	zoom into the selected area
[Shift + left click drag]	move to the specified offset
[Escape]	reinitialization
[I]	zoom in
[O]	zoom out
[T]	change number of threads (decrement with shift key)
[C]	change the number of iterations (decrement with shift key)
[P]	change active palette (decrement with shift key)
[A]	activate 5-point antialiasing of the image
[S]	smooth the transitions between iteration colors

It reports the computation time in the status bar below the fractal image, as shown below: